

NLU course project

Alessio Novel (256172)

University of Trento

alessio.novel@studenti.unitn.it

The following is the report for the first part of the project: the one about Language Modeling.

1. Introduction

This part of the project focused on implementing and improving a next-word prediction model. Performances were evaluated with the perplexity value (the lower the better).

When tuning the hyperparameters, I decided not to keep fixed the following parameters:

- **Patience:** The default value of 10 successfully stopped the training when the model started to overfit.
- **Batch size:** I decided to keep it constant to avoid increasing computational cost and to focus the comparison on the other hyperparameters.
- **Clip:** I did not see the need to change it.
- **Number of epochs:** In most cases early stopping was activated. In instances where it did not, I interpreted it as a sign of bad training behavior, so I changed other parameters.

The patience value was changed only with NT-AvSGD (more details later).

I also did not modify the number of layers, bidirectionality, and neither added additional regularization techniques nor optimizer techniques.

2. Implementation details

2.1. Part A

I implemented the RNN model [1] based on lab materials, to use it as Baseline.

For the first step, I simply replaced the RNN Pytorch implementation with the LSTM one [2], without changing anything else.

For the second step, I implemented the two dropout layers [3], calling the dropout right after the embedding layer and right before the last linear layer. I saw that there also was a dropout parameter directly in the LSTM module, but it does not work with 1 layer, so I implemented it by myself.

For the third step, I implemented AdamW [4] without changing anything else. AdamW is a smarter optimizer than SGD which uses a different adaptive learning rate for each weight and weight decay.

2.2. Part B

I copied the LSTM model from the first step of Part A, setting the embedding and the hidden size to match, to use it as Baseline.

For the first step, I implemented weight tying [5] [6]. This was done by simply equalizing the two weight matrices in the initialization of the model.

For the second step, I implemented the variational dropout [7]. Since I did not find any Pytorch implementation of a

dropout which maintains the same mask for each layer, I manually implemented it using LSTM cell [8] and writing all the logic as in the original code [9].

For the third step, I manually implemented the NT-AvSGD as in the pseudocode of the paper [9].

3. Results

3.1. Part A

For the baseline I found that decreasing the learning rate from 1 to 0.3 allows the training to be more precise and obtain the best result (obviously requiring more epochs). Going more down like 0.1 worsened the performance, probably because of local minima. Increasing hidden size didn't improve the results here.

Transitioning to LSTM significantly reduced perplexity (probably due to the known vanishing gradient problem of RNN). In this case a higher learning rate like 4 leads to better performance. At learning rate 5 performance was worse. Also in this case increasing the hidden size didn't bring improvements.

Dropout regularized successfully the model. In fact with dropout I was able to increase the hidden size (increasing also the dropout rate to regularize) reaching better results each time. At a certain point I stopped because the increasing of the hidden size was slowing down too much the training.

Replacing SGD with AdamW reached the same results but with way less epochs (with the default configuration). Tuning the learning rate a bit I obtained the best results in very few epochs compared to SGD.

In Table 1 I show the best results I obtained for each configuration.

3.2. Part B

For the baseline I made a little tuning increasing the hidden size and increasing the learning rate.

Weight Tying reduced the number of parameters and acted as a great regularization. But in this case I obtained the best result decreasing the hidden size.

Variational Dropout successfully regularized even more, beating the "standard" dropout of the previous experiment. Also in this case it was possible to increase the hidden size by increasing also the dropout rate.

NT-AvSGD did not change the training, but selecting as a final model an average of models yielded to the best results. In this case I changed a bit the n parameter and the patience in order to make the average between models that are more near the best one.

In Table 2 I show the best results I obtained for each configuration.

Table 1: *Best perplexity results for each configuration (Part A).*

Configuration	LR	Hidden Size	Dropout	Test PPL
Baseline RNN	0.3	200	-	155.03
Baseline LSTM	3.0	300	-	138.34
Dropout LSTM	2.5	800	0.5 / 0.6	110.08
AdamW LSTM	0.0005	800	0.5 / 0.6	104.10

Table 2: *Best perplexity results for each configuration (Part B).*

Configuration	LR	Hidden Size	Dropout	Test PPL
Baseline LSTM	2.0	500	-	134.49
Weight Tying LSTM	1.0	300	-	121.60
Variational Dropout LSTM	3.0	500	0.3	104.76
NT-AvSGD LSTM	3.0	600	0.4	98.61

4. Other experiments

In the folders you can find a file named `experiments_results.json` in which there are all the experiments I did in the order I did that shows the reasoning I have done in the finetuning part.

5. References

- [1] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, “Recurrent neural network based language model,” in *INTER-SPEECH 2010*, 2010, pp. 1045–1048.
- [2] PyTorch, “Long short-term memory (LSTM),” <https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html>.
- [3] —, “Dropout,” <https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html>.
- [4] —, “Adamw,” <https://docs.pytorch.org/docs/stable/generated/torch.nn.AdamW.html>.
- [5] O. Press and L. Wolf, “Using the output embedding to improve language models,” *arXiv preprint arXiv:1608.05859*, 2017.
- [6] Martinlwx, “An explanation of weight tying,” <https://martinlwx.github.io/en/an-explanation-of-weight-tying/>, 2023.
- [7] Y. Gal, “A theoretically grounded application of dropout in recurrent neural networks,” *arXiv preprint arXiv:1512.05287*, 2016.
- [8] PyTorch, “Long short-term memory cell (LSTMCell),” <https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTMCell.html>.
- [9] S. Merity, N. S. Keskar, and R. Socher, “Regularizing and optimizing LSTM language models,” in *International Conference on Learning Representations (ICLR)*, 2018, code available at <https://github.com/salesforce/awd-lstm-lm>.